PRÁCTICA CALIFICADA 1

Programar, validar y explicar

- **Curso:** Estructuras de Datos y Algoritmos
- **Modalidad:** Trabajo en parejas
- **Lenguaje:** C++17
- **Entorno de trabajo:** Visual Studio Code, GitHub, Copilot y Live Share
- **Escala de calificación:** Sobre 20 puntos
- **Fecha de publicación:** martes 31 de marzo, 2:00 p. m.
- **Fecha límite de entrega:** miércoles 8 de abril, 11:59 p. m.
- **Sustentación oral:** viernes 10 de abril, en horario de clase
- **Entrega:** Cada pareja deberá entregar la dirección del repositorio en GitHub + video de sustentación + README + reportes breves.

1. Presentación

La PC1 evalúa tres dimensiones del trabajo en programación:

1. **Algoritmo:** resolver un problema de manera correcta y justificar su complejidad.
2. **Ingeniería básica:** compilar, probar, sanitizar, organizar código y usar Git/GitHub.
3. **Evidencia experimental:** comparar builds, observar rendimiento y sustentar decisiones con datos básicos.

Esta práctica está alineada con los contenidos de **Semana 0**, **Proyecto 0** y **Semana 1**. En consecuencia, no basta con obtener una solución aceptada: la pareja debe **resolver, validar, analizar y sustentar** su trabajo.

2. Propósito

Al finalizar esta práctica, cada pareja deberá ser capaz de:

- Resolver un problema algorítmico en **C++17**
- Justificar la **correctitud**, la **finitud** y la **complejidad** de su solución
- Comparar su solución con al menos una alternativa más ingenua o menos eficiente;
- Usar herramientas de compilación, pruebas, sanitización, cobertura y profiling con g++ de manera elemental.
- Usar **Git y GitHub** como parte del proceso de trabajo
- Sustentar sus decisiones con evidencia conceptual, experimental y oral.

3. Organización del trabajo

- La práctica se realiza en **parejas**.
- Cada pareja resolverá **un problema distinto**.

- Ambos integrantes deben participar de manera visible en el desarrollo y en la sustentación.
- El trabajo debe quedar evidenciado en el historial del repositorio.

4. Problemas asignados

Cada pareja trabajará con uno de los siguientes problemas de LeetCode:

1. Two Sum
2. Valid Parentheses
3. Binary Search
4. Two Sum II - Input Array Is Sorted
5. Best Time to Buy and Sell Stock
6. Counting Bits
7. Maximum Subarray
8. Min Stack
9. Evaluate Reverse Polish Notation
10. Find Minimum in Rotated Sorted Array
11. Search in Rotated Sorted Array
12. Product of Array Except Self
13. Generate Parentheses
14. 3Sum
15. Longest Consecutive Sequence

Los problemas deben ser diferentes por pareja y será por orden de inscripción.

5. Enfoque de la evaluación

La PC1 no se entenderá solo como "resolver un problema de LeetCode y explicarlo", sino como: **Resolver, validar y analizar una solución algorítmica en C++17 con evidencia experimental básica y herramientas de compilación.**

La evaluación comprende dos bloques obligatorios:

A. Núcleo algorítmico

La pareja debe resolver el problema asignado y defender conceptualmente su solución.

B. Bloque experimental

La pareja debe analizar su solución usando herramientas básicas del curso: compilación, optimización, tests, sanitizers, cobertura y profiling. Cuando una técnica no sea natural para el problema principal, deberá presentarse un **microexperimento complementario**.

6. Cobertura conceptual de Semana 1

Además del funcionamiento del programa, cada pareja deberá vincular su solución con los conceptos fundamentales de **Semana 1**.

Fundamentos de algoritmos

- Entrada y salida del algoritmo
- Operaciones básicas del modelo RAM
- Determinismo
- Factibilidad
- Finitud
- Correctitud.

Razonamiento algorítmico

- Orden local y orden global, cuando aplique
- Invariantes
- Monotonicidad, cuando aplique
- Prueba de finitud
- Prueba de corrección.

Calidad de algoritmos

- Calidad práctica
- Degeneración
- Robustez
- Reutilización
- Eficiencia.

Complejidad y costo computacional

- Medición del costo computacional;
- Complejidad temporal
- Complejidad espacial
- Complejidad asintótica
- Notaciones O, Ω y Θ
- Mejor caso, peor caso y, cuando corresponda, caso promedio
- Jerarquía de crecimiento
- Tamaño de entrada.

Recursión

- Caso base

- Reducción del problema
- Trazado recursivo
- Patrones de recursión: lineal, divide y vencerás y múltiples ramas, cuando corresponda.

ADT y diseño

- ADT
- Interfaz e implementación
- Encapsulación.

Límites teóricos

- Computabilidad e intratabilidad como discusión conceptual básica, cuando corresponda al problema.

7. Trabajo a realizar

A. Núcleo algorítmico obligatorio

Cada pareja debe:

- Implementar una solución correcta en **C++17**
- Describir con claridad la **entrada**, la **salida** y la **especificación** del problema
- Identificar cuál es el **tamaño de entrada** relevante para el análisis del costo
- Justificar si el algoritmo es **determinista**
- Justificar la **factibilidad** de la solución propuesta
- Justificar la **finitud** del algoritmo
- Justificar la **correctitud** de la solución mediante uno o más de los siguientes recursos:
    - invariante,
    - argumento de progreso,
    - caso base e inducción,
    - razonamiento equivalente;
- Analizar:
    - complejidad temporal,
    - complejidad espacial,
    - mejor caso,
    - peor caso,
    - y, cuando tenga sentido, caso promedio;
- Ubicar su solución en la jerarquía de crecimiento relevante del curso
- Comparar la solución final con al menos **una alternativa más ingenua o menos eficiente**;
- Comentar la **robustez** de la solución frente a casos borde o entradas extremas
- Discutir la **reutilización** del código mediante funciones, módulos o encapsulación básica

- Y, cuando el problema lo permita, identificar nociones de **monotonicidad**, **orden local / orden global**, **recursión** o **ADT**.

B. Bloque experimental obligatorio

1. Compilación y optimización

La pareja debe compilar y probar su solución, como mínimo, en los siguientes builds:

Build de depuración

g++ -std=c++17 -O0 -g -Wall -Wextra -Wpedantic ...

Build depurable optimizado

g++ -std=c++17 -Og -g -Wall -Wextra -Wpedantic ...

Build release

g++ -std=c++17 -O2 -DNDEBUG ...

Build compacto

g++ -std=c++17 -Os ...

Además, se recomienda comparar también con:

g++ -std=c++17 -O1 ...
g++ -std=c++17 -O3 ...

Para estos builds, la pareja debe reportar observaciones sobre:

- Correctitud
- Tiempo total
- Tamaño del ejecutable
- Estabilidad entre corridas
- Facilidad de depuración
- Warnings relevantes, si aparecen.

2. Sanitizers

Es obligatorio ejecutar al menos una vez la solución con:

g++ -std=c++17 -O1 -g -fsanitize=address,undefined ...

Deben reportar:

- Si se detectó algún problema
- O qué tipo de errores habría podido detectar esta configuración en un programa mal escrito.

No se exige ThreadSanitizer en esta práctica, porque los problemas asignados no involucran concurrencia.

3. Cobertura

Deben repasar sobre cobertura y presentar:

- **Cobertura de líneas** sobre el conjunto de pruebas
- **Cobertura de condiciones** sobre una función pequeña o crítica.

La cobertura de caminos queda como exploración opcional o bono en funciones pequeñas.

Compilación sugerida:

```
g++ -std=c++17 --coverage -O0 ...
```

4. Profiling

Es necesario realizar al menos una ejecución con gprof:

```
g++ -std=c++17 -O2 -pg ...
```

La pareja debe indicar, de manera breve:

- Qué función concentra más tiempo
- Si eso coincide con lo esperado por el análisis de complejidad
- Qué conclusión extraen.

5. Benchmark básico

Cada pareja debe realizar un experimento simple, usando chrono o una técnica equivalente, para observar al menos una de estas comparaciones:

- Solución final vs solución ingenua;
- Una estructura de datos vs otra;
- Una variante con mejor idea algorítmica vs una menos eficiente;
- O una diferencia básica de compilación.

No se pide un benchmark profesional. Se pide un experimento pequeño, claro y razonado.

8. Microoptimización vs algoritmo

La práctica debe mostrar explícitamente que una mejora en la elección del algoritmo o de la estructura de datos suele tener más impacto que una microoptimización o un cambio de bandera de compilación.

Ejemplos válidos de comparación:

- Fuerza bruta vs hashing;
- Ordenamiento total vs estrategia más adecuada;
- Búsqueda lineal vs búsqueda binaria;
- Implementación ingenua vs una con mejor complejidad;
- push_back sin reserve() vs con reserve() en un microexperimento complementario.

Cuando el problema principal no se preste naturalmente a una de estas comparaciones, la pareja debe incluir un **microexperimento complementario** apoyado en el material del curso.

9. STL y rendimiento

No se exige que todos los problemas usen exactamente las mismas herramientas STL. En cambio, cada pareja deberá justificar uno de estos dos caminos:

Camino 1. Aplicado al problema principal

Usar o comparar una herramienta STL relevante para su solución.

Ejemplos posibles:

- sort
- lower_bound
- vector
- reserve()
- stack
- unordered_set
- **unordered_map.**

Camino 2. Experimento complementario

Si el problema no se presta naturalmente, la pareja debe desarrollar y explicar un microexperimento complementario relacionado con STL y rendimiento.

10. Estructura mínima del repositorio

```
pc1-parejaXX/
 README.md
 CMakeLists.txt
 src/
  main.cpp
  solution.cpp
  solution.hpp
 tests/
  tests.cpp
 bench/
  benchmark.cpp
 reports/
  optimization_report.md
  profiling_report.md
  coverage_report.md
```

Se permite usar:

- assert
- mini harness de pruebas
- CTest
- GoogleTest o Catch2 de manera opcional.

Para esta práctica, **assert + patrón AAA + CTest** son suficientes.

11. Pruebas

Pruebas obligatorias

La pareja debe incluir pruebas que cubran al menos:

- Casos normales
- Casos pequeños
- Casos borde
- Al menos un caso diseñado por la propia pareja
- Y, cuando sea viable, una comparación contra una solución de referencia o ingenua.

Enfoque de pruebas

Las pruebas deben:

- Usar assert o framework equivalente

- Seguir, de preferencia, el patrón **AAA** (Arrange, Act, Assert)
- Mencionar si se aplicó una idea de **TDD**
- Integrarse con CTest si se usa CMake.

12. Git y GitHub

En esta práctica es obligatorio usar **Git y GitHub**, pero con un flujo mínimo.

Comandos mínimos esperados

```
git clone URL
git status
git add .
git commit -m "mensaje claro"
git pull
git push
```

Opcional:

```
git log --oneline
```

Reglas mínimas

- Mínimo **4 commits por pareja**
- Mínimo **2 commits por estudiante**
- Ambos deben aparecer en el historial
- No se aceptará un único commit final.

No se exige, en esta primera práctica, el uso de branches ni pull requests.

Recursos sugeridos para aprender Git y GitHub

#	Tipo	Enlace	Descripción
1	**Interactivo (Mejor para empezar)**	Learn Git Branching (en español)	Juego interactivo visual. Altamente recomendable.
2	**GitHub Skills Oficiales**	skills.github.com	Cursos interactivos dentro de GitHub
3	**Introduction to Git (con VS Code)**	Introduction to Git	Curso práctico oficial
4	**Introduction to GitHub**	Introduction to GitHub	Pull Requests, Issues, etc.

5	**VS Code Oficial (Microsoft)**	Using Git with VS Code (YouTube)	8 minutos muy claros
6	**Tutorial en español con VS Code**	Git y GitHub con VS Code - Elliot Garamendi	Paso a paso práctico
7	**GitHub Pull Requests en VS Code**	GitHub Pull Requests and Issues	Extensión oficial

GitHub Docs -Hello World: https://docs.github.com/get-started/quickstart/hello-world

Pro Git Book (git-scm.com): https://git-scm.com/book/en/v2

13. Uso de Copilot y Live Share

Copilot

Está permitido, pero no reemplaza la comprensión.

En el video, la pareja debe mencionar:

- Una sugerencia de Copilot que aceptó
- Una sugerencia que rechazó
- y por qué.

Live Share

Está permitido y recomendado para el trabajo colaborativo.

Se sugiere alternar los roles:

- Una persona conduce
- La otra revisa
- Y luego cambian.

14. Video de sustentación

Duración: Entre **8 y 10 minutos; máximo 12 minutos**

Requisitos

- Deben aparecer **los dos integrantes**
- Ambos deben hablar
- La participación debe ser equilibrada
- La defensa debe mostrar comprensión real del código y de las decisiones tomadas.

Estructura sugerida

1. Presentación del problema.
2. Entrada, salida y especificación.
3. Idea general de la solución.
4. Correctitud, finitud e invariantes.
5. Complejidad temporal y espacial.
6. Comparación con una alternativa ingenua.
7. Robustez, casos borde y reutilización.
8. Tests, sanitizers y cobertura.
9. Builds y observaciones experimentales.
10. Profiling y benchmark básico.
11. Git/GitHub y organización del trabajo.
12. Uso de Copilot y reflexión final.

Preguntas obligatorias

La pareja debe responder en el video, como mínimo, las siguientes preguntas:

- ¿Cuál es la **entrada**, la **salida** y la **especificación** del problema?
- ¿Cuál es el **tamaño de entrada** relevante para el análisis?
- ¿Cuál fue la estrategia más simple que pensaron primero?
- ¿Por qué la solución final es mejor?
- ¿El algoritmo es **determinista**? ¿Por qué?
- ¿Cómo justifican la **factibilidad** de la solución?
- ¿Cómo saben que el algoritmo termina?
- ¿Cómo saben que la solución es correcta?
- ¿Qué **invariante**, propiedad de progreso o argumento de corrección usaron?
- ¿Existe alguna idea de **monotonicidad** u orden útil en el problema?
- ¿Cuál es la complejidad temporal?
- ¿Cuál es la complejidad espacial?
- ¿Cuál es el mejor caso, el peor caso y, si aplica, el caso promedio?
- ¿En qué parte de la jerarquía O(1), O(log n), O(n), O(n^2), O(2^n) ubican su solución?
- ¿Qué caso borde fue importante?
- ¿Qué riesgos de **degeneración** o peor comportamiento identificaron?
- ¿Qué tan **robusta** es la solución ante entradas extremas?
- ¿Qué parte del diseño favorece la **reutilización** o la encapsulación?
- Si el problema usa recursión: ¿cuál es el caso base, cómo se reduce el problema y qué patrón de recursión aparece?
- Si el problema usa una estructura abstracta: ¿cuál es el **ADT**, cuál es la interfaz y cuál es la implementación?
- ¿Qué cambió entre -O0, -Og, -O2 y -Os?
- ¿Qué mejoró más: la bandera de compilación o el algoritmo?
- ¿Qué mostró gprof?

- ¿Qué detectó ASan o UBSan, o qué habrían detectado?
- ¿Qué mostró la cobertura y qué faltó cubrir?
- ¿Qué estructura de datos o herramienta STL fue decisiva?
- ¿Qué aprendieron de Git/GitHub al realizar la práctica?
- ¿Qué relación tiene su problema con la idea de **computabilidad** o **intratabilidad**, aunque sea de forma conceptual básica?

15. README obligatorio

El README.md debe incluir, como mínimo:

- Nombres completos de ambos integrantes
- Problema asignado
- Enlace al enunciado de LeetCode
- Especificación breve del problema
- Entrada y salida
- Tamaño de entrada relevante
- Descripción de la solución
- Justificación de determinismo y factibilidad
- Argumento de finitud
- Argumento de correctitud
- Invariante, cuando aplique
- Monotonicidad, cuando aplique
- Complejidad temporal
- Complejidad espacial
- Mejor, peor y promedio cuando aplique
- Ubicación aproximada en la jerarquía de crecimiento del curso
- Alternativa ingenua o menos eficiente
- Discusión breve de robustez, degeneración y reutilización
- Instrucciones para compilar
- Instrucciones para ejecutar
- Instrucciones para correr tests
- Breve descripción del benchmark realizado
- Tabla resumida de builds y resultados
- Observaciones de sanitizers
- Resumen de cobertura
- Resumen de profiling
- Discusión sobre microoptimización vs algoritmo
- Qué se usó de Copilot y cómo se validó.

16. Entregables

Cada pareja deberá entregar la dirección del:

1. **Repositorio GitHub** con el código fuente, pruebas, benchmark y reportes.
2. **Video de sustentación** de 8 a 10 minutos; máximo 12 minutos.
3. **README** completo.
4. **Reportes breves** de optimización, profiling y cobertura.

La sustentación se evalúa individualmente dentro del trabajo en pareja.

17. Rúbrica

Dado que la práctica se realiza fuera del aula y se permite el uso de IA, la **comprensión demostrada en la sustentación oral tendrá el mayor peso**.

La calificación es **sobre 20 puntos**.

Defensa oral = 12 puntos

3 puntos - Claridad al explicar

- Explicación ordenada
- Uso correcto del lenguaje técnico
- Capacidad de comunicar ideas sin limitarse a leer el código.

4 puntos - Dominio conceptual de Semana 1

- Entrada y salida
- Tamaño de entrada
- Determinismo
- Factibilidad
- Finitud
- Correctitud
- Invariantes
- Monotonicidad, cuando aplique
- Complejidad temporal y espacial
- Notaciones O, Ω y Θ
- Mejor, peor y caso promedio cuando corresponda
- Recursión, ADT, interfaz e implementación, cuando apliquen
- Robustez, degeneración, reutilización y eficiencia.

2 puntos - Participación equilibrada de ambos

- Ambos estudiantes deben intervenir de manera sustancial;
- Ambos deben explicar partes distintas del trabajo.

3 puntos - Respuestas individuales

- Cada estudiante debe responder al menos una pregunta técnica distinta con solvencia.

Correctitud y calidad del código = 4 puntos

- Solución correcta
- Organización básica razonable
- Claridad de funciones
- Manejo de casos borde
- Uso adecuado de C++17.

Tests, sanitizers, cobertura y experimento básico = 2.5 puntos

- Pruebas relevantes
- Ejecución con ASan y UBSan
- Cobertura mínima solicitada
- Benchmark o comparación experimental clara.

Uso de Git/GitHub = 0.75 puntos

- Commits suficientes
- Commits de ambos
- Mensajes razonables
- Historial consistente con el proceso.

README y reportes técnicos = 0.75 puntos

- Claridad
- Instrucciones completas
- Resultados bien resumidos
- Decisiones justificadas.

18. Evidencia de entrega, sustentación e integridad académica

La evaluación de esta práctica requiere evidencia verificable tanto del **resultado** como del **proceso de trabajo**. Por ello, la calificación no dependerá únicamente de que el programa funcione, sino también de que la pareja pueda demostrar autoría, comprensión y participación real en el desarrollo.

18.1. Repositorio obligatorio

El repositorio GitHub forma parte obligatoria de la entrega.

En él debe encontrarse, como mínimo, la evidencia del trabajo realizado por la pareja:

- Código fuente
- Pruebas
- Benchmark o experimento básico

- README
- Reportes solicitados
- Historial de commits.

Si una pareja **no presenta repositorio dentro del plazo establecido**, la práctica se considerará **incompleta** y no podrá recibir puntaje en los criterios que dependan de esa evidencia, incluyendo:

- Correctitud y calidad del código
- Tests
- Sanitizers
- Cobertura
- Experimento básico
- Uso de Git/GitHub.

El docente podrá evaluar únicamente aquello que sí sea verificable con la evidencia efectivamente entregada.

18.2. Sustentación oral obligatoria

La sustentación oral es obligatoria y forma parte central de la evaluación. Cada estudiante debe poder explicar y defender el trabajo presentado. Si un estudiante **no asiste a la sustentación** sin justificación aceptada conforme a las normas del curso o de la institución, perderá el puntaje correspondiente a la **defensa oral individual**.

Si existiera evidencia objetiva de participación técnica en el repositorio, dicho estudiante podrá conservar únicamente el puntaje de los componentes no orales que sí puedan verificarse de manera razonable.

18.3. Inasistencia de uno de los integrantes

Si asiste solo uno de los integrantes de la pareja, se evaluará oralmente únicamente al estudiante presente. El integrante ausente recibirá **cero en el componente oral**, salvo que presente una justificación aceptada conforme a las reglas del curso o de la institución.

La asistencia de un integrante no reemplaza la obligación del otro de sustentar su propia participación y comprensión del trabajo.

18.4. Comprensión del código entregado

El código entregado debe poder ser explicado por ambos integrantes. La evaluación no se limita a verificar si el programa produce una salida correcta. También se evaluará si la pareja comprende:

- La estrategia utilizada
- La correctitud de la solución

- La finitud del algoritmo
- La complejidad temporal y espacial
- Los casos borde
- Y las decisiones de implementación tomadas.

Si durante la sustentación oral la pareja no logra justificar su solución, sus pruebas o sus decisiones principales, la nota de la práctica podrá verse restringida, aunque el programa funcione correctamente.

Si el repositorio no compila o la solución no es funcional, la nota máxima posible no podrá superar el puntaje de los componentes efectivamente verificables.

18.5. Autoría académica y uso de herramientas de apoyo

El uso de herramientas de apoyo, incluyendo asistentes de programación o herramientas de inteligencia artificial, deberá ajustarse a lo permitido expresamente en esta práctica. En todos los casos, cada estudiante debe comprender y poder explicar el código entregado como propio.

El problema no es el uso instrumental de una herramienta, sino presentar como propio un código cuya elaboración, comprensión o validación no pueda sostenerse académicamente.

18.6. Indicios de autoría no verificable

Si durante la revisión del repositorio, del código o de la sustentación existen indicios de que la autoría académica no puede verificarse adecuadamente, el docente podrá solicitar evidencia adicional, por ejemplo:

- Explicación detallada de partes clave del código
- Modificación en vivo de una parte de la solución
- Justificación de estructuras de datos y algoritmos elegidos
- Historial de commits
- Pruebas intermedias
- Versiones previas
- Comparación con alternativas más simples o menos eficientes.

La presencia de estilos, comentarios, patrones de redacción, marcas textuales inusuales o fragmentos que parezcan generados automáticamente **no será considerada por sí sola prueba suficiente** de falta académica.

La decisión se basará en el conjunto de evidencias disponibles y en la capacidad real del estudiante para sustentar el trabajo presentado.

18.7. Falta de acreditación de comprensión o autoría

Si un estudiante no puede explicar de manera satisfactoria el funcionamiento, la correctitud, la complejidad o decisiones básicas del código presentado como propio, el docente podrá considerar que la autoría académica o la comprensión del trabajo **no han quedado suficientemente acreditadas**.

En ese caso, los criterios afectados podrán recibir calificación **parcial o nula**, sin perjuicio de la aplicación de las normas de integridad académica establecidas por la institución.

18.8. Justificaciones y casos excepcionales

Los casos de inasistencia o incumplimiento por motivos de salud, fuerza mayor u otras circunstancias excepcionales serán tratados conforme a las normas del curso y de la institución, siempre que exista la justificación correspondiente dentro del plazo establecido.

19. Criterios de descuento

Se descontará puntaje si ocurre alguno de estos casos:

- El repositorio no compila o no existe
- La solución no funciona correctamente
- Solo una persona habla en el video
- No hay evidencia de trabajo de ambos
- No hay tests suficientes
- No hay evidencia de sanitizers
- No hay reporte básico de cobertura o profiling
- No hay historial de commits suficiente
- El README está incompleto
- El video solo lee el código sin justificarlo
- La pareja no puede explicar partes esenciales del repositorio.

20. Consideraciones finales

Esta práctica no busca solamente "llegar al accepted". Busca que ustedes puedan:

- Programar
- Probar
- Justificar
- Medir
- Usar herramientas reales
- Y comunicar ideas técnicas con claridad.

Se evaluará tanto el resultado como el proceso.